Universidad del Valle de Guatemala

Facultad de Ingenieria - Departamento de Ciencias de la Computacion

Segundo semestre 2026

Laboratorio 11 - MLOps aplicado al Observatorio de Inteligencia Turistica

Trabajo final: MLflow, aceptacion, tablero, monitoreo y Docker

Integrantes

Javier Lopez - 23415

Pedro Avila - 23089

Brandon Rivera - 23088

Guatemala, 1 de noviembre de 2026

1. Objetivo y alcance

El Observatorio de Inteligencia Turistica necesita actualizar sus estimaciones de forma controlada y reproducible, y poder responder en todo momento que datos, que codigo y que configuracion produjeron la cifra que consulta un directivo. El avance entrego el alcance, el diagrama, un pipeline ejecutable con un solo comando y los seis controles de calidad. Este trabajo final completa el flujo:

- Registro de cada ejecucion en MLflow Tracking (codigo, fuentes, parametros, metricas, controles y artefactos), incluidas las corridas detenidas o rechazadas.
- Regla de aceptacion formal sobre 57 meses de backtesting con origen movil y comparacion sistematica contra dos procedimientos de referencia.
- Integracion con el tablero del Laboratorio 10: nueva seccion Estimaciones oficiales que solo lee la version aprobada.
- Monitoreo de actualizacion, calidad y resultados, y restauracion documentada de versiones anteriores.
- Contenerizacion con Docker Compose (MLflow, pipeline y tablero) con persistencia, reproducible desde una copia limpia del repositorio.

La variable, la fuente, las columnas, la clave de unicidad y los periodos se mantienen como se definieron en el avance: movimientos mensuales de viajeros (personas), serie flujo_mensual_nacional, Base de Migracion 2009-2026 en su version mensual, clave (periodo, destino).

2. Diagrama del flujo final

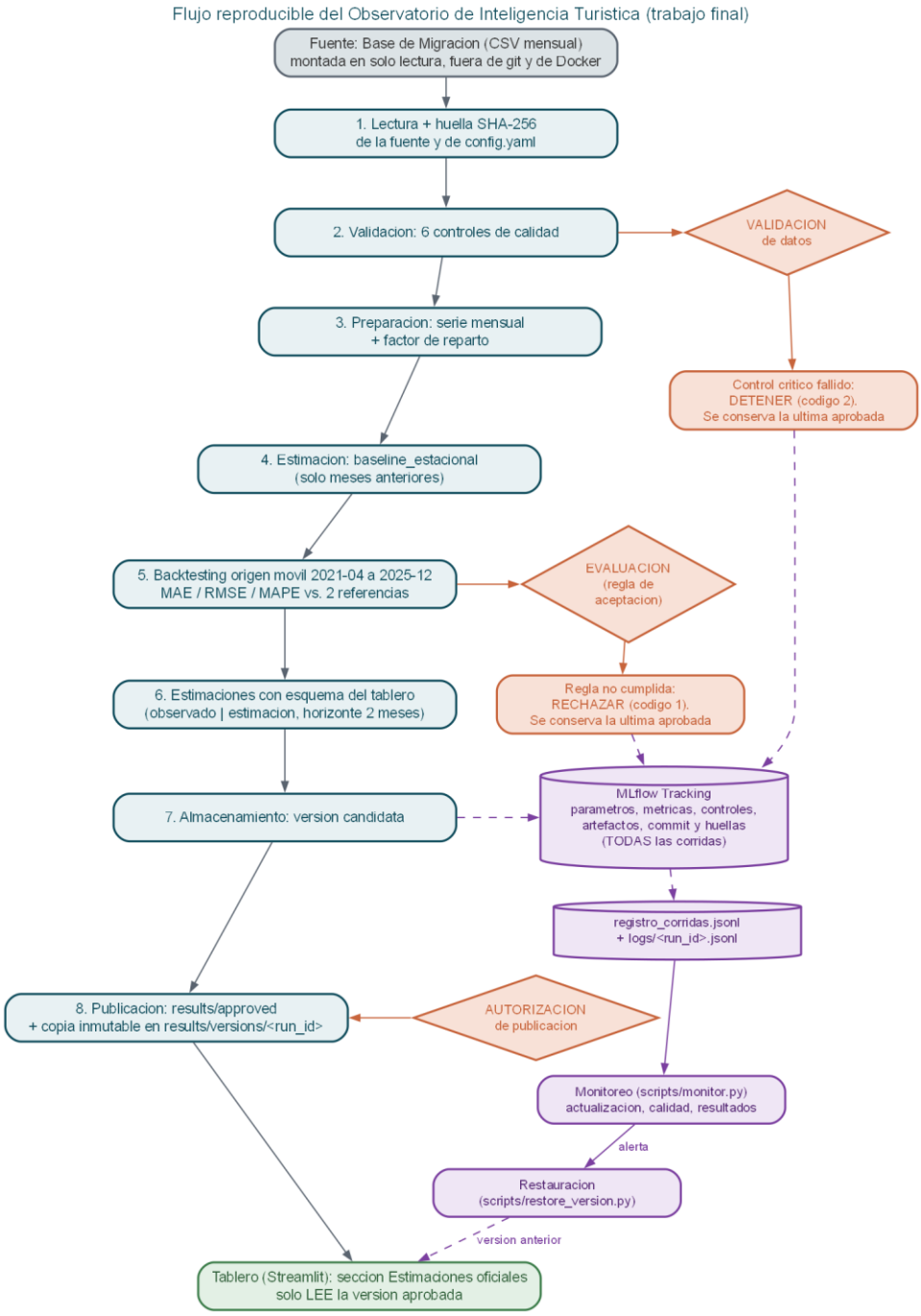


Figura 1. Ocho etapas automatizadas, tres puntos de control (validacion, evaluacion y autorizacion) y los componentes de registro, monitoreo y restauracion.

3. Pipeline reproducible

Todo el flujo se ejecuta con un solo comando y toda ruta, periodo, parametro y umbral vive en config/config.yaml:

```
python -m src.run_pipeline --config config/config.yaml
docker compose up --build -d        # flujo completo en contenedores
```

Etapa	Responsabilidad
1. Lectura	Carga la fuente y calcula la huella SHA-256 de la fuente y de la configuracion
2. Validacion	Seis controles; si uno critico falla se detiene (codigo 2)
3. Preparacion	Serie mensual y factor de reparto (supuesto documentado)
4. Estimacion	baseline_estacional: nivel de 12 meses por indice estacional, solo con meses anteriores
5. Evaluacion	Backtesting 2021-04 a 2025-12 y regla de aceptacion
6. Estimaciones	Tabla con el esquema del tablero: observados, meses ocultos y 2 meses futuros
7. Almacenamiento	Version candidata con reporte de calidad, backtest y metadatos
8. Publicacion	Solo si la regla autoriza; copia inmutable en results/versions/<run_id>

Medidas de reproducibilidad: dependencias con version fijada (requirements.txt y dashboard/requirements.txt), semilla 42, imagen base python:3.12-slim, commit de Git y huellas registrados en cada corrida, escritura atomica de los archivos publicados y codigos de salida documentados (0 aprobada, 1 rechazada por la regla, 2 detenida por control critico, 3 error). Dos corridas equivalentes producen la misma huella de fuente, la misma huella de configuracion y las mismas metricas (seccion 9).

4. Controles automaticos de calidad

#	Control	Advertencia	Detiene
1	Columnas y tipos	-	Falta una columna o tipo incompatible
2	Fechas y cobertura	Periodo sin datos o cobertura < 20 dias	Fecha invalida
3	Unicidad	-	Duplicado por (periodo, destino)
4	Valores faltantes	NA en dias_fuente	NA en periodo, destino o viajeros
5	Valores y denominadores	-	Negativos, inf/NaN o dias_fuente = 0
6	Regla metodologica	-	Algun mes proviene de 'diaria_agregada'

Respecto del avance se corrigio el control 5: ante una columna ausente fallaba por una excepcion interna; ahora reporta el problema de forma explicita. Los periodos sin cobertura nunca se convierten en cero.

5. Evaluacion y regla de aceptacion

El procedimiento se evalua con origen movil sobre 57 meses (2021-04 a 2025-12): cada mes se estima solo con los meses anteriores, como se haria en produccion. Se compara contra el ultimo valor observado y el promedio historico sobre los mismos meses.

Procedimiento	MAE	RMSE	MAPE	Meses
baseline_estacional (oficial)	44,941	61,838	15.6 %	57
Referencia: ultimo valor	49,947	64,004	16.9 %	57
Referencia: promedio historico	53,666	72,888	16.3 %	57

El procedimiento oficial se equivoca 10 % menos que la mejor referencia. Sobre los dos meses ocultos que ve el tablero (2025-11 y 2025-12) el MAE es 10,414 frente a 65,872 y 89,432. Enero y febrero de 2026 se publican como estimacion sin error calculable, porque no hay observados comparables.

Regla de aceptacion (verificable y previa a publicar). Una candidata se publica solo si se cumplen todas las condiciones:

- Ningun control critico de calidad fallo.
- La salida esta completa: ninguna estimacion faltante.
- El backtesting evaluo al menos 24 meses.
- MAE <= 120,000 y RMSE <= 160,000 personas/mes (umbrales fijados segun la escala de la variable).
- El MAE no es peor que el de la mejor referencia simple.

Si la regla no se cumple, la candidata queda en results/candidate para inspeccion, la corrida se registra con sus razones y la version vigente del tablero no cambia.

6. Registro en MLflow Tracking

src/tracking.py registra cada corrida en el experimento observatorio_flujo_mensual_nacional. Se registran todas, tambien las detenidas (estado FAILED) y las rechazadas, porque son las que permiten monitorear la calidad en el tiempo.

Elemento	Que se registra
Codigo	Commit de Git (tag mlflow.source.git.commit) y version del procedimiento
Fuentes	Ruta, huella SHA-256 y si es la muestra sintetica
Parametros	Metodo, ventana, factor de reparto, periodos, umbrales y semilla
Metricas	MAE, RMSE y MAPE del backtesting y de los meses ocultos (oficial y referencias), conteo de controles, aprobado, codigo de salida
Controles	Resumen como metricas y reporte completo como artefacto
Artefactos	config.yaml, reporte_calidad.json, estimaciones.csv, backtest.csv, run_metadata.json y log de la corrida

En local el registro usa sqlite (mlruns/mlflow.db). En Docker Compose el pipeline apunta al servicio mlflow (MLFLOW_TRACKING_URI=http://mlflow:5000), que guarda base y artefactos en un volumen persistente. El identificador de MLflow se anota en los metadatos de la version publicada, y el tablero lo muestra.

7. Integracion con el tablero

El tablero del Laboratorio 10 se incorporo al repositorio (carpeta dashboard/) con una nueva seccion Estimaciones oficiales. La seccion solo lee results/approved: cambiar un filtro nunca vuelve a ajustar el modelo.

- Encabezado: fecha de actualizacion, version del procedimiento, MAE de validacion, proxima estimacion y estado de la version.
- V21: observado (linea continua azul) frente a estimacion (linea punteada morada), con el observado real de los meses ocultos para contrastar.
- V22: backtesting del procedimiento frente a las referencias, con tabla de MAE, RMSE y MAPE.
- V23: estado de los seis controles, resultado del monitoreo y trazabilidad (run, commit, huellas e identificador de MLflow).
- Advertencias de limitaciones: uso de la muestra sintetica, version restaurada, meses sin error calculable.
- Sin version aprobada muestra "Estimaciones no disponibles" en lugar de ceros.

8. Monitoreo y recuperacion

Dimension	Alerta cuando
Actualizacion	La version aprobada tiene mas de 45 dias o no existe
Calidad	Hay 2 o mas corridas seguidas sin aprobar (controles o regla)
Resultados	El MAE empeora mas de 25 % respecto de la aprobada anterior

scripts/monitor.py escribe results/monitoreo.json (lo muestra el tablero) y sale con codigo 1 si hay alerta, para poder usarse en un cron o en CI. Cada version aprobada se guarda de forma inmutable en results/versions/<run_id>/; scripts/restore_version.py la vuelve a publicar sin reajustar nada y deja constancia en el registro de corridas y en results/approved/restauracion.json.

```
python scripts/monitor.py
python scripts/restore_version.py --listar
python scripts/restore_version.py --anterior --motivo "..."
```

9. Evidencia de ejecucion

Secuencia ejecutada desde cero (salidas completas en docs/evidencia/): dos corridas equivalentes, falla critica, falla de la regla, restauracion, recuperacion y monitoreo.

Corrida	Estado	Codigo	MAE
20261102T060112552	aprobado	0	44,941
20261102T060118512	aprobado	0	44,941
20261102T060123988	detenido_por_control_critico	2	-
20261102T060129573	rechazado_por_regla	1	325,431
20261102T060112552	restaurado	-	-
20261102T060134547	aprobado	0	44,941

- Corridas equivalentes: misma huella de fuente (ec60d714ed6df774), misma huella de configuracion y metricas identicas; ambas registradas en MLflow.
- Columna ausente: los controles 1 y 5 fallan, el pipeline se detiene con codigo 2 y la version aprobada no cambia.
- Metrica degradada (datos validos pero erraticos): los controles pasan, pero la regla rechaza (MAE y RMSE sobre el umbral), codigo 1, sin publicar.
- Restauracion de la version anterior y, al corregir los datos, recuperacion con una nueva version aprobada. El monitoreo termina sin alertas.
- Docker Compose: el mismo flujo se verifico en contenedores, y los volumenes de MLflow y de versiones persistieron tras docker compose down / up.
- 19 pruebas automaticas (pytest) cubren controles, publicacion, equivalencia, detencion, rechazo, restauracion y monitoreo.

10. Contenerizacion

Servicio	Funcion	Persistencia
mlflow	Servidor de MLflow Tracking (puerto 5000)	Volumen ots_mlflow
pipeline	Corre el pipeline y el monitoreo; termina	Volumenes ots_resultados y ots_logs
dashboard	Tablero Streamlit (puerto 8501), solo lectura	Lee ots_resultados

Los datos institucionales no se copian en ninguna imagen (.dockerignore); se montan en solo lectura desde data/institucional. El servidor MLflow reutiliza la imagen del pipeline para usar exactamente la misma version de MLflow. La version aprobada versionada en Git siembra el volumen la primera vez, para que el tablero tenga algo que mostrar desde el inicio.

```
git clone <repo> && cd lab11-data
docker compose up --build -d
docker compose run --rm pipeline       # nueva actualizacion
```

11. Limitaciones

- La version publicada en el repositorio se genero con la muestra sintetica: ilustra el flujo, no es dato oficial. Con la base institucional montada el flujo es el mismo.
- El procedimiento es un baseline estacional; no incorpora rutas nuevas, crisis ni campanas de promocion.
- El error del backtesting es mayor que el de los meses ocultos porque incluye la recuperacion pospandemia y el cambio de clasificacion de 2023.
- Los umbrales de aceptacion y de monitoreo son decisiones del equipo segun la escala de la variable; estan en config.yaml y se pueden auditar y ajustar.

Referencias

MLflow. (2026). MLflow Tracking. https://mlflow.org/docs/latest/ml/tracking

Docker. (2026). Docker Compose. https://docs.docker.com/compose/

Streamlit. (2026). Streamlit documentation. https://docs.streamlit.io/

INGUAT / IGM. Base de Migracion 2009 - junio 2026 (datos compartidos con fines academicos).